

SonarQube Setup and Code Analysis on Ubuntu 20.04

Objective

This document explains:

- * Install SonarQube on Ubuntu 20.04
- * Configure PostgreSQL database
- * Configure SonarQube service
- * Start and access SonarQube UI
- * Install Sonar Scanner
- * Clone Java project from Git
- * Build Maven project
- * Analyze source code using SonarQube

1. Update Ubuntu Server:

```
sudo apt update -y  
sudo apt install unzip wget curl vim git -y
```

2. Install Java 17

Install OpenJDK 17:

```
sudo apt install openjdk-17-jdk -y
```

```
java -version
```

Set JAVA_HOME:

```
echo "export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64" >> ~/.bashrc  
echo "export PATH=$JAVA_HOME/bin:$PATH" >> ~/.bashrc  
source ~/.bashrc
```

```
echo $JAVA_HOME
```

3. Install PostgreSQL

```
sudo apt install postgresql postgresql-contrib -y
```

Enable and start service:

```
sudo systemctl enable postgresql  
sudo systemctl start postgresql  
sudo systemctl status postgresql
```

4. Create SonarQube Database

Login to PostgreSQL:

```
sudo -u postgres psql
```

Create database and user:

```
sql  
CREATE USER sonar WITH ENCRYPTED PASSWORD 'sonar123';  
CREATE DATABASE sonarqube OWNER sonar;  
GRANT ALL PRIVILEGES ON DATABASE sonarqube TO sonar;  
\q
```

5. Linux Kernel Tuning

```
sudo vim /etc/sysctl.conf
```

#Add:

```
vm.max_map_count=262144  
fs.file-max=65536
```

#Apply changes:

```
sudo sysctl -p
```

#Update limits:

```
sudo vim /etc/security/limits.conf
```

#Add:

```
sonarqube - nofile 65536  
sonarqube - nproc 4096
```

6. Download SonarQube

```
cd /opt
```

```
sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-9.9.6.92038.zip
```

#Extract package:

```
sudo unzip sonarqube-9.9.6.92038.zip  
sudo mv sonarqube-9.9.6.92038 sonarqube
```

7. Create SonarQube User

```
sudo groupadd sonar  
sudo useradd -d /opt/sonarqube -g sonar sonar  
sudo chown -R sonar:sonar /opt/sonarqube
```

8. Configure SonarQube Database

Edit configuration:

```
sudo vim /opt/sonarqube/conf/sonar.properties
```

```
properties  
sonar.jdbc.username=sonar  
sonar.jdbc.password=sonar123  
sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube  
sonar.web.port=9000
```

9. Configure SonarQube Service User

Edit startup file:

```
sudo vim /opt/sonarqube/bin/linux-x86-64/sonar.sh
```

Find:

```
#RUN_AS_USER=
```

Update:

```
RUN_AS_USER=sonar
```

10. Create Systemd Service

Create service file:

```
sudo vim /etc/systemd/system/sonarqube.service
```

Add:

```
ini
```

```
[Unit]
```

```
Description=SonarQube service
```

```
After=syslog.target network.target
```

```
[Service]
```

```
Type=forking
```

```
ExecStart=/opt/sonarqube/bin/linux-x86-64/sonar.sh start
```

```
ExecStop=/opt/sonarqube/bin/linux-x86-64/sonar.sh stop
```

```
User=sonar
```

```
Group=sonar
```

```
Restart=always
```

```
LimitNOFILE=65536
```

```
LimitNPROC=4096
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable sonarqube
```

```
sudo systemctl start sonarqube
```

```
sudo systemctl status sonarqube
```

```
netstat -tunlp | grep 9000
```

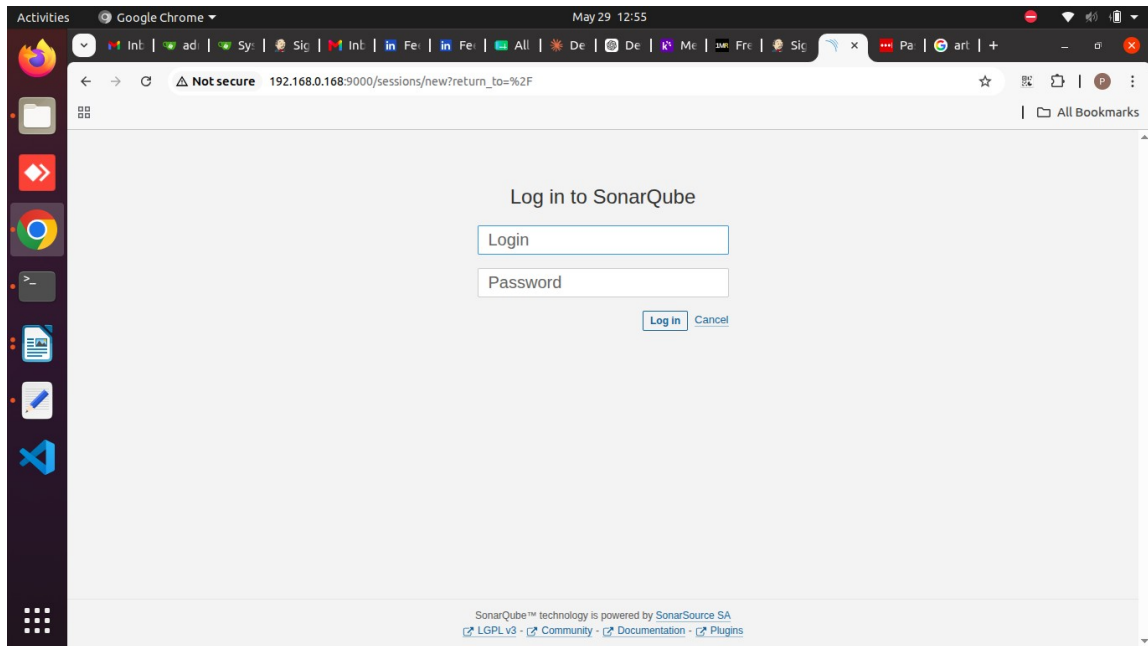
11. Access SonarQube UI

#Open browser: http://SERVER_IP:9000

Username: admin

Password: admin

Change password after login.



12. Install Sonar Scanner

`cd /opt`

Download scanner:

```
sudo wget https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-scanner-cli-7.1.0.4889-linux-x64.zip
```

```
sudo unzip sonar-scanner-cli-7.1.0.4889-linux-x64.zip
```

```
sudo mv sonar-scanner-7.1.0.4889-linux-x64 sonar-scanner
```

Create symbolic link:

```
sudo ln -s /opt/sonar-scanner/bin/sonar-scanner /usr/local/bin/sonar-scanner
```

```
sonar-scanner --version
```

13. Install Maven

```
sudo apt install maven -y
```

```
mvn -version
```

14. Clone Project from Git

```
git clone https://github.com/username/project.git
```

```
cd project
```

15. Configure Maven Authentication (Private Repository)

Create Maven settings file:

```
mkdir -p ~/.m2
```

```
vim ~/.m2/settings.xml
```

Add:

```
xml
```

```
<settings>
```

```
<servers>
```

```
<server>
```

```
<id>central</id>
```

```
<username>YOUR_USERNAME</username>
```

```
<password>YOUR_PASSWORD</password>
</server>
</servers>
</settings>
```

Note:

- * Replace credentials with actual Artifactory/Nexus credentials.
- * Match <id> with repository ID from pom.xml.

16. Build Java Project

Compile project:

mvn clean package -DskipTests

##If successful, compiled classes will be generated under:
target

17. Generate SonarQube Token

Inside SonarQube UI:

My Account

→ Security

→ Generate Tokens

Copy generated token.

18. Run SonarQube Analysis

Run scan command:-

sonar-scanner \

-Dsonar.projectKey=clouds-project-mtn\

-Dsonar.projectName=clouds-project-mtn \

-Dsonar.sources=src/main/java \

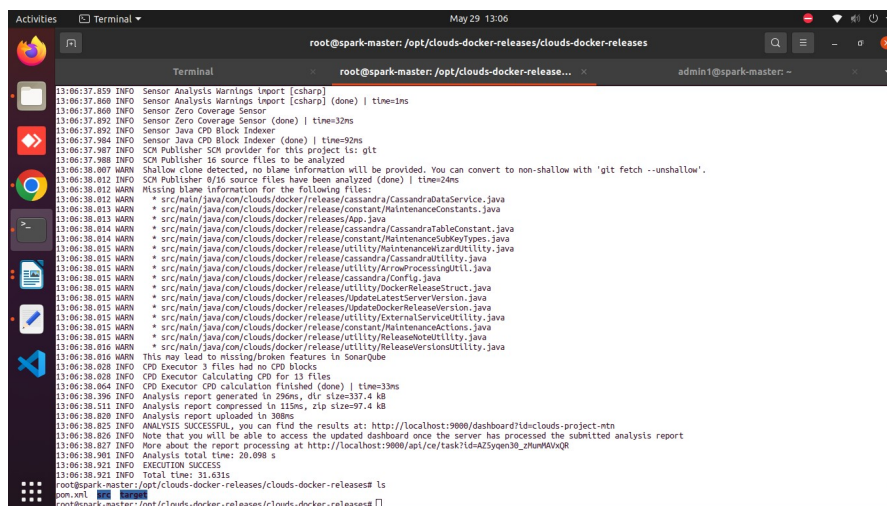
-Dsonar.java.binaries=target \

-Dsonar.host.url=http://localhost:9000 \

-Dsonar.login=YOUR_TOKEN

Expected Output:

ANALYSIS SUCCESSFUL



```
root@spark-master: /opt/clouds-docker-releases/clouds-docker-releases
root@spark-master: /opt/clouds-docker-release...
admin1@spark-master: ~
13:06:37.859 INFO Sensor Analysis Warnings Import [csharp]
13:06:37.860 INFO Sensor Analysis Warnings Import [csharp] (done) | time=1ms
13:06:37.860 INFO Sensor Zero Coverage Sensor
13:06:37.892 INFO Sensor Zero Coverage Sensor (done) | time=32ms
13:06:37.892 INFO Sensor Java CPD Block Indexer
13:06:37.964 INFO Sensor Java CPD Block Indexer (done) | time=92ms
13:06:37.967 INFO SCR Publisher SCR provider for this project is: git
13:06:37.988 INFO SCR Publisher 16 source files to be analyzed
13:06:38.007 WARN Shallow clone detected, no blame information will be provided. You can convert to non-shallow with 'git fetch --unshallow'.
13:06:38.012 INFO SCR Publisher 8/16 source files have been analyzed (done) | time=24ms
13:06:38.012 WARN Missing blame information for the following files:
13:06:38.012 WARN * src/main/java/com/clouds/docker/release/cassandra/CassandraDataService.java
13:06:38.013 WARN * src/main/java/com/clouds/docker/release/App.java
13:06:38.014 WARN * src/main/java/com/clouds/docker/release/cassandra/CassandraTableConstant.java
13:06:38.014 WARN * src/main/java/com/clouds/docker/release/constant/MaintenanceSubKeyTypes.java
13:06:38.015 WARN * src/main/java/com/clouds/docker/release/utility/MaintenanceCassandraUtility.java
13:06:38.015 WARN * src/main/java/com/clouds/docker/release/cassandra/CassandraUtility.java
13:06:38.015 WARN * src/main/java/com/clouds/docker/release/utility/ArrowProcessingUtil.java
13:06:38.015 WARN * src/main/java/com/clouds/docker/release/cassandra/Config.java
13:06:38.015 WARN * src/main/java/com/clouds/docker/release/utility/DockerReleaseStruct.java
13:06:38.015 WARN * src/main/java/com/clouds/docker/release/UpdateAttestServerVersion.java
13:06:38.015 WARN * src/main/java/com/clouds/docker/release/utility/ExternalServiceUtility.java
13:06:38.015 WARN * src/main/java/com/clouds/docker/release/constant/MaintenanceActions.java
13:06:38.015 WARN * src/main/java/com/clouds/docker/release/utility/ReleaseMilestoneUtility.java
13:06:38.016 WARN * src/main/java/com/clouds/docker/release/utility/ReleaseVersionsUtility.java
13:06:38.016 WARN This may lead to missing/broken features in SonarQube
13:06:38.028 INFO CPD Executor 3 files had no CPD blocks
13:06:38.028 INFO CPD Executor Calculating CPD for 13 files
13:06:38.064 INFO CPD Executor CPD calculation finished (done) | time=33ms
13:06:38.396 INFO Analysis report generated in 296ms, dir size=337.4 KB
13:06:38.511 INFO Analysis report compressed in 115ms, zip size=97.4 KB
13:06:38.820 INFO Analysis report uploaded in 308ms
13:06:38.825 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dashboard?id=clouds-project-mtn
13:06:38.826 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
13:06:38.827 INFO Here about the report processing at http://localhost:9000/api/ce/task?id=A25gen30_0nuW6wXq
13:06:38.901 INFO Analysis total time: 20.098 s
13:06:38.921 INFO EXECUTION SUCCESS
13:06:38.921 INFO Total time: 31.431s
root@spark-master: /opt/clouds-docker-releases/clouds-docker-releases# ls
pom.xml
root@spark-master: /opt/clouds-docker-releases/clouds-docker-releases#
```

Activities Google Chrome May 29 12:54

Not secure 192.168.0.168:9000/projects

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration

Search for projects...

My Favorites All

Filters

Quality Gate

Passed 1 Failed 0

Reliability (Bugs)

A rating 0 B rating 0 C rating 0 D rating 1 E rating 0

Security (Vulnerabilities)

A rating 1 B rating 0 C rating 0 D rating 0

Search by project name or key

1 project(s) Perspective: Overall Status Sort by: Name

Create Project

clouds-project-mtn Passed Last analysis: 1 day ago

Bugs Vulnerabilities Hotspots Reviewed Code Smells Coverage Duplications Lines

0.0% 0.0% 1.9k Java

1 of 1 shown

SonarQube™ technology is powered by SonarSource SA
Community Edition - v9.9.6 (build 92038) NO LONGER ACTIVE LGPL v3 Community Documentation Plugins Web API

Activities Google Chrome May 29 12:54

Not secure 192.168.0.168:9000/dashboard?id=clouds-project-mtn

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration

clouds-project-mtn main

Last analysis of this Branch had 4 warnings May 28, 2026 at 12:25 PM Version not provided

Overview Issues Security Hotspots Measures Code Activity Project Settings Project Information

To benefit from more of SonarQube's features, set up analysis in your favorite CI.

QUALITY GATE STATUS

Passed All conditions passed.

MEASURES

New Code Overall Code

1 Bugs Reliability D

0 Vulnerabilities Security A

5 Security Hotspots 0.0% Reviewed Security Review E

Activities Google Chrome May 29 12:54

Not secure 192.168.0.168:9000/quality_gates/show/AZ5tLn8vaRCN0779rDq

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration

Quality Gates Create

Sonar way DEFAULT BUILT-IN Copy

This quality gate complies with Clean as You Code

This quality gate complies with the Clean as You Code methodology, so that you benefit from the most efficient approach to delivering Clean Code. It ensures that:

- No new bugs are introduced
- No new vulnerabilities are introduced
- All new security hotspots are reviewed
- New code has limited technical debt
- New code has limited duplication
- New code is properly covered by tests

Conditions

Conditions on New Code

Metric	Operator	Value
Coverage	is less than	80.0%
Duplicated Lines (%)	is greater than	3.0%
Maintainability Rating	is worse than	A (Technical debt ratio is less than 5.0%)
Reliability Rating	is worse than	A (No bugs)

19. SonarQube Dashboard Explanation

Quality Gate

Shows whether project passed quality checks.

* Passed = deployment allowed

* Failed = deployment blocked

Bugs

Reliability issues in code.

Vulnerabilities

Security-related coding problems.

Security Hotspots

Security-sensitive code requiring manual review.

Code Smells

Maintainability issues and bad coding practices.

Coverage

Unit test coverage percentage.

Duplication

Duplicate code percentage.

20. Common Useful Commands

Restart SonarQube:

```
sudo systemctl restart sonarqube
```

```
tail -f /opt/sonarqube/logs/sonar.log
```

Check Elasticsearch logs:

```
tail -f /opt/sonarqube/logs/es.log
```

Scanner debug mode:

```
sonar-scanner -X
```

21. Typical CI/CD Flow

Developer Push Code

↓

GitHub / GitLab

↓

Jenkins Pipeline

↓

Maven Build

↓

SonarQube Analysis

↓

Quality Gate Check

↓

Deploy to Production

22. Recommended Enhancements

- * Integrate with Jenkins
- * Add JaCoCo coverage reports
- * Enable pull request analysis
- * Configure Quality Gates
- * Integrate with GitHub/GitLab

23. Final Verification

Open SonarQube Dashboard: http://SERVER_IP:9000

Verify:

- * Project visible
- * Analysis completed
- * Issues displayed
- * Quality Gate status available

SonarQube setup and code analysis completed successfully.

NOTE: You can create your own artifact server (maven), then your settings.xml will be like below:- (Replace your server url and username Password)

cat settings.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<settings xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.1.0
http://maven.apache.org/xsd/settings-1.1.0.xsd"
xmlns="http://maven.apache.org/SETTINGS/1.1.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <servers>
    <server>
      <username>maven-admin</username>
      <password>U66Me6<,TyV3</password>
      <id>central</id>
    </server>
  </servers>
  <profiles>
    <profile>
      <repositories>
        <repository>
          <snapshots>
<updatePolicy>always</updatePolicy>
          <enabled>true</enabled>

          </snapshots>
          <id>central</id>
          <name>plugins-release</name>
          <url>http://10.13.X.X:9095/artifactory/plugins-release</url>
        </repository>
      </repositories>
    </profile>
  </profiles>
</settings>
```

```

<repository>
  <snapshots>
    <enabled>>false</enabled>
  </snapshots>
  <id>paho-mqtt-client</id>
  <name>Paho MQTT Client</name>
  <url>https://repo.eclipse.org/content/repositories/paho-releases/</url>
</repository>
</repositories>
<pluginRepositories>
  <pluginRepository>
    <snapshots>
<updatePolicy>always</updatePolicy>
    <enabled>true</enabled>
    </snapshots>
    <id>central</id>
    <name>plugins-release</name>
    <url>http://10.13.X.X:9095/artifactory/plugins-release</url>
  </pluginRepository>
<pluginRepository>
  <snapshots>
    <enabled>>false</enabled>
  </snapshots>
  <id>paho-mqtt-client</id>
  <name>Paho MQTT Client</name>
  <url>https://repo.eclipse.org/content/repositories/paho-releases/</url>
</pluginRepository>
</pluginRepositories>
<id>artifactory</id>
</profile>
</profiles>
<activeProfiles>
  <activeProfile>artifactory</activeProfile>
</activeProfiles>
</settings>
<!--<project>
...
<repositories>
  <repository>
    <id>sonar-maven-plugin</id>
    <url>https://plugins.sonarsource.com/maven/releases/</url>
  </repository>
</repositories>
...
</project>-->

```